

# 13 запросов варианта: подробный разбор SQL

Информационная система городской телефонной сети

Каждый запрос — построчно: что, зачем и как

---

## Содержание

|    |                                                                 |   |
|----|-----------------------------------------------------------------|---|
| 1  | Как устроены все 13 запросов (общее) .....                      | 2 |
| 2  | Фундамент: представления и функция .....                        | 2 |
| 3  | Мини-справочник по SQL .....                                    | 3 |
| 4  | Q1. Абоненты АТС .....                                          | 4 |
| 5  | Q2. Свободные номера .....                                      | 4 |
| 6  | Q3. Должники .....                                              | 4 |
| 7  | Q4. Рейтинг АТС по долгам .....                                 | 5 |
| 8  | Q5. Таксофоны и общественные телефоны .....                     | 5 |
| 9  | Q6. Доля простых и льготных .....                               | 6 |
| 10 | Q7. Абоненты с параллельными телефонами .....                   | 6 |
| 11 | Q8. Телефоны по адресу .....                                    | 7 |
| 12 | Q9. Город — лидер по межгороду .....                            | 7 |
| 13 | Q10. Полная информация по номеру .....                          | 8 |
| 14 | Q11. Спаренные, которые можно распарить .....                   | 8 |
| 15 | Q12. Внутренние номера с малым числом внешних звонков .....     | 8 |
| 16 | Q13. Должники: кого уведомить / отключить / заблокировать ..... | 9 |

# 1 Как устроены все 13 запросов (общее)

Все аналитические запросы лежат в одном файле `backend/server/src/analytics.rs`. Каждый — это маленький обработчик HTTP-запроса: берёт параметры из строки URL, подставляет их в SQL и отдаёт результат как JSON-массив.

**Где смотреть.** Запросы: `backend/server/src/analytics.rs`. Представления и функция, на которых они стоят: `backend/migrations/0009_views.sql`. Привязка «запрос → таблицы»: таблица в конце `docs/schema.md`.

Во всех запросах повторяются три приёма — разберём их один раз, чтобы потом не повторяться.

## 1.1 Приём 1. Результат сразу как JSON (`wrap`)

Любой `SELECT` оборачивается функцией `wrap`, чтобы PostgreSQL сам собрал все строки в один JSON-массив:

```
fn wrap(inner: &str) -> String {
    format!("SELECT coalesce(json_agg(row_to_json(q)), '[]'::json) FROM ({inner}) q")
}
```

`row_to_json(q)` превращает строку в объект `{...}`, `json_agg(...)` собирает их в массив `[ {...}, {...} ]`, `coalesce(..., '[]')` отдаёт пустой массив, если строк нет. Благодаря этому Rust-код просто забирает одно готовое JSON-значение и возвращает его в ответ — не нужно описывать структуру под каждый запрос.

## 1.2 Приём 2. «Выключаемый» фильтр (`$1 IS NULL OR ...`)

Почти все параметры необязательные. Чтобы один запрос обслуживал и «по конкретной АТС», и «по всей сети», используется такая конструкция:

```
AND ($1::bigint IS NULL OR pbx_id = $1)
```

Если параметр `$1` не передан (`NULL`) — левая часть `IS NULL` истинна, и всё условие истинно (фильтр «выключен»). Если передан — работает правая часть `pbx_id = $1`. Так не нужно плодить десяток почти одинаковых запросов.

## 1.3 Приём 3. Параметры и приведение типов

`$1`, `$2`, ... — это **параметры** (подставляются драйвером отдельно от текста — защита от SQL-инъекций). Запись `$1::bigint`, `$2::text` — это **приведение типа** (`cast`): мы заранее говорим, какого типа параметр. А `category::text` приводит `enum`-колонку к тексту, чтобы сравнить со строковым параметром.

# 2 Фундамент: представления и функция

Чтобы не повторять одни и те же `JOIN`-ы в каждом запросе, заранее созданы **представления** (сохранённые запросы, к которым обращаются как к таблице).

## 2.1 `v_subscriber_full` — «плоский» абонент

Склеивает абонента с его номером, АТС и адресом и добавляет вычисленный возраст:

```
CREATE VIEW v_subscriber_full AS
SELECT s.id, s.last_name, s.first_name, s.gender, s.birth_date,
       date_part('year', age(s.birth_date))::int AS age,
       s.category, s.privilege, s.status,
       pn.number, pn.line_type, pn.intercity,
       p.id AS pbx_id, p.name AS pbx_name, p.pbx_type, p.district AS pbx_district,
       a.district, a.street, a.house, a.apartment
FROM subscriber s
JOIN phone_number pn ON pn.id = s.phone_number_id
JOIN pbx p          ON p.id = pn.pbx_id
JOIN address a      ON a.id = s.address_id;
```

Используется в запросах 1, 3, 6, 7, 10, 13. `age` считается из даты рождения функцией `age()` + `date_part('year', ...)`.

## 2.2 v\_subscriber\_debt — долг абонента

Считает долг каждого абонента, разбитый на три части (абонплата / межгород / пени):

```
CREATE VIEW v_subscriber_debt AS
SELECT s.id AS subscriber_id,
       COALESCE(sub.amt,0) AS subscription_debt,
       COALESCE(inter.amt,0) AS intercity_debt,
       COALESCE(pen.amt,0) AS penalty_debt,
       COALESCE(sub.amt,0)+COALESCE(inter.amt,0)+COALESCE(pen.amt,0) AS total_debt,
       LEAST(sub.oldest, inter.oldest) AS oldest_due_date
FROM subscriber s
LEFT JOIN (SELECT subscriber_id, sum(amount) amt, min(due_date) oldest
           FROM invoice WHERE kind='subscription' AND status IN ('pending','overdue')
           GROUP BY subscriber_id) sub ON sub.subscriber_id = s.id
LEFT JOIN (/* то же для kind='intercity' */) inter ON inter.subscriber_id = s.id
LEFT JOIN (SELECT subscriber_id, sum(amount) amt FROM penalty WHERE NOT paid
           GROUP BY subscriber_id) pen ON pen.subscriber_id = s.id;
```

Каждый подзапрос суммирует неоплаченные счета (status IN ('pending','overdue')) по виду. COALESCE(...,0) — если долгов нет, подставить 0 (а не NULL). oldest\_due\_date — самый ранний срок оплаты (для расчёта просрочки). Используется в запросах 3, 4, 13.

## 2.3 v\_pbx\_stats — сводка по АТС

По каждой АТС считает свободные номера, всего номеров и число абонентов:

```
count(pn.id) FILTER (WHERE pn.status = 'free') AS free_numbers,
count(pn.id) AS total_numbers,
count(DISTINCT s.id) AS subscribers
```

FILTER (WHERE ...) — посчитать только подходящие. Используется в запросе 11.

## 2.4 fn\_subscriber\_monthly\_fee(id) — абонплата

Функция: берёт тариф по (тип линии × открыт ли межгород) и применяет скидку 50% льготнику. Возвращает сумму абонплаты. Используется при подключении и в личном кабинете.

# 3 Мини-справочник по SQL

| Конструкция                 | Смысл                                                        |
|-----------------------------|--------------------------------------------------------------|
| JOIN t ON ...               | приклеить связанную таблицу (только совпавшие строки)        |
| LEFT JOIN t ON ...          | приклеить, сохранив все левые строки; нет пары → справа NULL |
| WHERE                       | фильтр строк (до группировки)                                |
| GROUP BY x                  | сгруппировать строки по x                                    |
| count(*), sum(c)            | агрегаты: число строк / сумма по группе                      |
| count(*) FILTER (WHERE ...) | посчитать только подходящие внутри группы                    |
| HAVING ...                  | фильтр уже по группам (после агрегации)                      |
| COALESCE(x, 0)              | заменить NULL на 0                                           |
| NULLIF(x, 0)                | превратить 0 в NULL (защита от деления на ноль)              |
| ILIKE 'Ив%'                 | регистронезависимый поиск по префиксу                        |
| EXISTS (...)                | есть ли хотя бы одна строка в подзапросе                     |
| CURRENT_DATE - d            | разница дат в днях                                           |
| col::text                   | привести значение (напр. enum) к тексту                      |

## 4 Q1. Абоненты АТС

**Задача варианта.** Получить перечень и общее число абонентов указанной АТС: полностью, только льготников, по возрастному признаку, по группе фамилий.

Параметры: pbx\_id, category (regular/privileged), min\_age, max\_age, surname (префикс).

```
SELECT * FROM v_subscriber_full
WHERE ($1::bigint IS NULL OR pbx_id = $1)
      AND ($2::text IS NULL OR category::text = $2)
      AND ($3::int IS NULL OR age >= $3)
      AND ($4::int IS NULL OR age <= $4)
      AND ($5::text IS NULL OR last_name ILIKE $5 || '%')
ORDER BY last_name, first_name;
```

Разбор:

- База — v\_subscriber\_full, где уже есть ФИО, номер, АТС, адрес и вычисленный age.
- (\$1 ... pbx\_id = \$1) — «указанная АТС». Не передали — значит по всей ГТС.
- (\$2 ... category::text = \$2) — «только льготники» (privileged) или простые (regular).
- age >= \$3 и age <= \$4 — «по возрастному признаку» (диапазон).
- last\_name ILIKE \$5 || '%' — «по группе фамилий»: \$5='Ив' найдёт Иванов, Иванова, Иващенко (|| '%' — любое окончание).
- ORDER BY last\_name, first\_name — по алфавиту.

**Что вернёт.** Массив абонентов. «Общее число» — это длина массива (фронт показывает счётчик).

## 5 Q2. Свободные номера

**Задача варианта.** Получить перечень и общее число свободных телефонных номеров на указанной АТС, по всей ГТС, по признаку возможности установки телефона в данном районе.

Параметры: pbx\_id, district.

```
SELECT pn.id, pn.number, pn.line_type, pn.intercity, p.name AS pbx_name, p.district
FROM phone_number pn JOIN pbx p ON p.id = pn.pbx_id
WHERE pn.status = 'free'
      AND ($1::bigint IS NULL OR pn.pbx_id = $1)
      AND ($2::text IS NULL OR p.district = $2)
ORDER BY pn.number;
```

Разбор:

- pn.status = 'free' — свободный номер (без абонента; статус поддерживается триггером автоматически).
- JOIN pbx — чтобы вернуть имя и район АТС.
- pbx\_id — на конкретной АТС; пусто — по всей ГТС; district — по району.
- Наличие свободных номеров в районе и есть «признак возможности установки телефона».

## 6 Q3. Должники

**Задача варианта.** Должники на указанной АТС, по всей ГТС, по району; те, кто должен уже больше недели (месяца); по признаку долга за межгород и/или абонплату; по размеру долга.

Параметры: pbx\_id, district, min\_days, kind (any/subscription/intercity), min\_amount.

```
SELECT vf.last_name, vf.number, vf.pbx_name,
       d.subscription_debt, d.intercity_debt, d.total_debt,
       (CURRENT_DATE - d.oldest_due_date) AS days_overdue
FROM v_subscriber_debt d JOIN v_subscriber_full vf ON vf.id = d.subscriber_id
```

```

WHERE d.total_debt > 0
AND ($1::bigint IS NULL OR vf.pbx_id = $1)
AND ($2::text IS NULL OR vf.pbx_district = $2)
AND ($3::int IS NULL OR (CURRENT_DATE - d.oldest_due_date) >= $3)
AND ($5::numeric IS NULL OR d.total_debt >= $5)
AND ($4::text IS NULL OR $4 = 'any'
    OR ($4 = 'subscription' AND d.subscription_debt > 0)
    OR ($4 = 'intercity' AND d.intercity_debt > 0))
ORDER BY d.total_debt DESC;

```

Разбор:

- v\_subscriber\_debt d даёт суммы долга, JOIN v\_subscriber\_full vf — ФИО/номер/АТС.
- d.total\_debt > 0 — оставить только должников.
- pbx\_id / pbx\_district — «на АТС / по району»; пусто — по всей ГТС.
- days\_overdue = CURRENT\_DATE - oldest\_due\_date (просрочка в днях); >= \$3 — «больше недели/месяца» (передаём 7 или 30).
- total\_debt >= \$5 — «по размеру долга».
- Блок \$4 — «по признаку долга»: subscription (есть долг по абонплате), intercity (по межгороду), any или пусто — любой.
- ORDER BY total\_debt DESC — крупнейшие должники сверху.

**Что вернёт.** На демо-данных: Кузнецов — 1250 ₽ (просрочка 47 дн.), Васильева — 100 ₽ (17 дн.).

## 7 Q4. Рейтинг АТС по долгам

**Задача варианта.** Определить АТС (любого или конкретного типа), на которой самое большое (маленькое) число должников и самая большая сумма задолженности.

Параметры: pbx\_type.

```

SELECT p.id AS pbx_id, p.name AS pbx_name, p.pbx_type::text AS pbx_type,
       count(d.subscriber_id) AS debtors,
       COALESCE(sum(d.total_debt), 0) AS debt_sum
FROM pbx p
LEFT JOIN phone_number pn ON pn.pbx_id = p.id
LEFT JOIN subscriber s ON s.phone_number_id = pn.id
LEFT JOIN v_subscriber_debt d ON d.subscriber_id = s.id AND d.total_debt > 0
WHERE ($1::text IS NULL OR p.pbx_type::text = $1)
GROUP BY p.id
ORDER BY debt_sum DESC;

```

Разбор:

- Начинаем с pbx и идём цепочкой LEFT JOIN: АТС → её номера → абоненты → их долг (только total\_debt > 0).
- Почему LEFT JOIN, а не JOIN: чтобы в рейтинг попали **все** АТС, даже те, где должников нет (у них debtors = 0).
- count(d.subscriber\_id) — число должников (значения NULL, появившиеся из-за LEFT JOIN, в count(колонка) не считаются — это и даёт ноль для «чистых» АТС).
- sum(d.total\_debt) — суммарный долг по АТС.
- pbx\_type — ограничить конкретным типом АТС.
- ORDER BY debt\_sum DESC — сверху АТС-лидер по долгу, снизу — минимум.

## 8 Q5. Таксофоны и общественные телефоны

**Задача варианта.** Получить перечень и число общественных телефонов и таксофонов во всём городе, принадлежащих указанной АТС, по признаку нахождения в данном районе.

Параметры: pbx\_id, district, kind (public/payphone).

```

SELECT pp.id, pp.kind::text AS kind, p.name AS pbx_name,
       a.district, a.street, a.house, pp.active
FROM public_phone pp
JOIN pbx p      ON p.id = pp.pbx_id
JOIN address a  ON a.id = pp.address_id
WHERE ($1::bigint IS NULL OR pp.pbx_id = $1)
      AND ($2::text IS NULL OR a.district = $2)
      AND ($3::text IS NULL OR pp.kind::text = $3)
ORDER BY pp.kind, a.district, a.street;

```

Разбор:

- public\_phone хранит вид (public — общественный, payphone — таксофон), его АТС и адрес.
- JOIN pbx и JOIN address — чтобы вернуть имя АТС и адрес (улица/дом/район).
- Фильтры: pbx\_id — «принадлежащих указанной АТС»; district — «в данном районе»; kind — отдельно таксофоны или общественные; всё пусто — по всему городу.

## 9 Q6. Доля простых и льготных

**Задача варианта.** Найти процентное соотношение обычных и льготных абонентов на указанной АТС, по всей ГТС, по данному району, по типам АТС.

Параметры: pbx\_id, district, pbx\_type.

```

SELECT count(*) AS total,
       count(*) FILTER (WHERE category = 'regular') AS regular,
       count(*) FILTER (WHERE category = 'privileged') AS privileged,
       round(100.0 * count(*) FILTER (WHERE category='regular') / NULLIF(count(*),0), 2)
AS regular_pct,
       round(100.0 * count(*) FILTER (WHERE category='privileged') / NULLIF(count(*),0), 2)
AS privileged_pct
FROM v_subscriber_full
WHERE ($1::bigint IS NULL OR pbx_id = $1)
      AND ($2::text IS NULL OR district = $2)
      AND ($3::text IS NULL OR pbx_type::text = $3);

```

Разбор:

- count(\*) — всего абонентов в выбранной области.
- count(\*) FILTER (WHERE category='regular') — сколько простых; аналогично — льготных.
- Процент:  $100.0 * \text{простых} / \text{всего}$ , округление round(..., 2) до 2 знаков.
- NULLIF(count(\*), 0) — если абонентов нет (0), превращаем делитель в NULL, чтобы не было **деления на ноль** (результат станет NULL, а не ошибка).
- Фильтры pbx\_id / district / pbx\_type задают область: АТС / район / тип АТС / вся ГТС.

**Что вернёт.** Одна строка, напр.: total=9, regular=7 (77.78%), privileged=2 (22.22%).

## 10 Q7. Абоненты с параллельными телефонами

**Задача варианта.** Абоненты, имеющие параллельные телефоны (по АТС / ГТС / району / типам АТС); только льготники с параллельными.

Параметры: pbx\_id, district, pbx\_type, privileged\_only (bool).

```

SELECT * FROM v_subscriber_full
WHERE line_type = 'parallel'
      AND ($1::bigint IS NULL OR pbx_id = $1)
      AND ($2::text IS NULL OR district = $2)
      AND ($3::text IS NULL OR pbx_type::text = $3)
      AND ($4::bool IS NULL OR $4 = false OR category = 'privileged')
ORDER BY number, last_name;

```

Разбор:

- `line_type = 'parallel'` — берём только абонентов на параллельных линиях.
- Обычные фильтры по АТС / району / типу АТС.
- `($4 IS NULL OR $4 = false OR category='privileged')` — если `privileged_only=true`, оставить **ТОЛЬКО ЛЬГОТНИКОВ** если `false` или не передан — всех.

## 11 Q8. Телефоны по адресу

**Задача варианта.** Определить, есть ли по данному адресу телефон; общее количество телефонов и/или количество с выходом на межгород, с открытым выходом на межгород — в данном доме, на конкретной улице.

Параметры: `district`, `street`, `house`.

```
SELECT a.district, a.street, a.house,
       count(pn.id) AS phones,
       count(pn.id) FILTER (WHERE pn.intercity IN ('open','closed')) AS with_intercity,
       count(pn.id) FILTER (WHERE pn.intercity = 'open') AS with_open_intercity
FROM phone_number pn JOIN address a ON a.id = pn.address_id
WHERE ($1::text IS NULL OR a.district = $1)
      AND ($2::text IS NULL OR a.street = $2)
      AND ($3::text IS NULL OR a.house = $3)
GROUP BY a.district, a.street, a.house
ORDER BY a.district, a.street, a.house;
```

Разбор:

- `JOIN address` — каждый номер привязан к адресу установки.
- `GROUP BY district, street, house` — группируем по **дому**.
- `phones = count(*)` — всего телефонов в доме (если  $> 0$  — телефон по адресу есть).
- `with_intercity` — у скольких есть выход на межгород: `intercity IN ('open','closed')` (есть техническая возможность — открыт или закрыт).
- `with_open_intercity` — у скольких межгород именно открыт.
- Фильтры: указать дом (`house`) → одна строка; указать только улицу → по всем домам улицы.

## 12 Q9. Город — лидер по межгороду

**Задача варианта.** Определить город, с которым происходит большее количество междугородных переговоров.

Без параметров.

```
SELECT c.name AS city, count(*) AS calls
FROM call_record cr JOIN city c ON c.id = cr.dest_city_id
WHERE cr.call_type = 'intercity'
GROUP BY c.name
ORDER BY calls DESC;
```

Разбор:

- `call_record` — журнал всех звонков; `JOIN city` — приклеить город назначения.
- `WHERE call_type = 'intercity'` — только междугородние.
- `GROUP BY c.name + count(*)` — для каждого города посчитать число звонков.
- `ORDER BY calls DESC` — первая строка и есть город-лидер.

**Что вернёт.** На демо-данных: Москва — 6 звонков (лидер), Екатеринбург — 1, Новосибирск — 1.

## 13 Q10. Полная информация по номеру

**Задача варианта.** Получить полную информацию об абонентах с заданным телефонным номером.

Параметр: number (обязательный).

```
SELECT * FROM v_subscriber_full WHERE number = $1;
```

Разбор:

- Простой запрос к v\_subscriber\_full по номеру.
- На один номер может приходиться **несколько** абонентов (параллельный/спаренный телефон) — тогда вернётся несколько строк, по строке на каждого, со всей информацией (ФИО, адрес, АТС, тип линии, межгород).

## 14 Q11. Спаренные, которые можно расспарить

**Задача варианта.** Получить перечень спаренных телефонов, для которых есть техническая возможность заменить их на обычные (выделить дополнительный номер).

Параметр: pbx\_id.

```
SELECT pn.id, pn.number, p.name AS pbx_name, p.district,
       st.free_numbers, p.free_channels
FROM phone_number pn
JOIN pbx p ON p.id = pn.pbx_id
JOIN v_pbx_stats st ON st.pbx_id = p.id
WHERE pn.line_type = 'paired'
      AND st.free_numbers > 0 AND p.free_channels > 0
      AND ($1::bigint IS NULL OR pn.pbx_id = $1)
ORDER BY pn.number;
```

Разбор:

- line\_type = 'paired' — берём спаренные номера.
- «Техническая возможность» = на их АТС есть, что выделить: st.free\_numbers > 0 (свободный номер, из v\_pbx\_stats) и p.free\_channels > 0 (свободный канал).
- Если оба условия выполнены — спаренный телефон можно «расспарить», выделив второму абоненту отдельный номер.

## 15 Q12. Внутренние номера с малым числом внешних звонков

**Задача варианта.** Получить перечень и число внутренних номеров определённой ведомственной или учрежденческой АТС, с которых за некоторый период было сделано менее определённого числа внешних звонков.

Параметры: pbx\_id, from, to (период), max\_calls (N).

```
SELECT pn.id, pn.number, p.name AS pbx_name, p.pbx_type::text AS pbx_type,
       count(cr.id) AS external_calls
FROM phone_number pn
JOIN pbx p ON p.id = pn.pbx_id AND p.pbx_type IN ('departmental', 'institutional')
LEFT JOIN call_record cr ON cr.from_number_id = pn.id AND cr.call_type = 'external'
      AND ($2::timestampz IS NULL OR cr.started_at >= $2::timestampz)
      AND ($3::timestampz IS NULL OR cr.started_at < $3::timestampz)
WHERE ($1::bigint IS NULL OR pn.pbx_id = $1)
GROUP BY pn.id, pn.number, p.name, p.pbx_type
HAVING count(cr.id) < $4
ORDER BY external_calls, pn.number;
```

Разбор:

- JOIN pbx ... AND p.pbx\_type IN ('departmental', 'institutional') — берём номера только **замкнутых** (ведомственных/учрежденных) АТС.
- LEFT JOIN call\_record на внешние звонки (call\_type='external') в периоде. LEFT — чтобы номера **без** звонков тоже попали (у них count = 0).
- GROUP BY по номеру + count(cr.id) — сколько внешних звонков у каждого номера.
- HAVING count(cr.id) < \$4 — оставить тех, у кого внешних звонков **меньше** N. HAVING — это фильтр уже по группам (после подсчёта), в отличие от WHERE.

**Важно.** Условия на период стоят **внутри** LEFT JOIN (после ON), а не в WHERE. Это важно: если поставить условие на правую таблицу (call\_record) в WHERE, то LEFT JOIN фактически превратится в обычный JOIN, и номера без звонков потеряются. В ON — фильтруем звонки, но строки-номера сохраняем.

## 16 Q13. Должники: кого уведомить / отключить / заблокировать

**Задача варианта.** Должники на АТС / по всей ГТС / по району, которым следует послать письменное уведомление, отключить телефон и/или выход на межгород.

Параметры: pbx\_id, district.

```
SELECT vf.last_name, vf.number, vf.pbx_name, vf.status, vf.intercity,
       d.subscription_debt, d.intercity_debt, d.total_debt,
       (d.subscription_debt > 0 AND NOT EXISTS (
         SELECT 1 FROM notification n WHERE n.subscriber_id = vf.id
         AND n.kind='subscription_debt' AND NOT n.resolved)) AS notice_subscription,
       (d.intercity_debt > 0 AND NOT EXISTS (
         SELECT 1 FROM notification n WHERE n.subscriber_id = vf.id
         AND n.kind='intercity_debt' AND NOT n.resolved)) AS notice_intercity,
       (d.subscription_debt > 0 AND vf.status <> 'disconnected' AND EXISTS (
         SELECT 1 FROM notification n WHERE n.subscriber_id = vf.id
         AND n.kind='subscription_debt' AND NOT n.resolved
         AND n.deadline < CURRENT_DATE)) AS should_disconnect,
       (d.intercity_debt > 0 AND vf.intercity = 'open' AND EXISTS (
         SELECT 1 FROM notification n WHERE n.subscriber_id = vf.id
         AND n.kind='intercity_debt' AND NOT n.resolved
         AND n.deadline < CURRENT_DATE)) AS should_block_intercity
FROM v_subscriber_debt d JOIN v_subscriber_full vf ON vf.id = d.subscriber_id
WHERE d.total_debt > 0
      AND ($1::bigint IS NULL OR vf.pbx_id = $1)
      AND ($2::text IS NULL OR vf.pbx_district = $2)
ORDER BY d.total_debt DESC;
```

Разбор. Для каждого должника считаются 4 булевых флага-рекомендации (через EXISTS / NOT EXISTS — «существует ли уведомление с такими условиями»):

- **notice\_subscription** — есть долг по абонплате И ещё **нет** активного уведомления (NOT EXISTS ... NOT resolved) → нужно **послать письменное уведомление**.
- **notice\_intercity** — то же для долга по межгороду.
- **should\_disconnect** — есть долг по абонплате, абонент ещё не отключён, И уже есть уведомление с **истёкшим** сроком (deadline < CURRENT\_DATE) → пора **отключать телефон**.
- **should\_block\_intercity** — есть долг по межгороду, межгород ещё открыт, И уведомление просрочено → пора **блокировать межгород**.

Это реализует всю логику задания: «при неуплате после письменного уведомления в течение N суток — отключение»; запрос находит, на каком этапе каждый должник.

**Что вернёт.** На демо-данных: Кузнецов → should\_disconnect = true, notice\_intercity = true (по межгороду уведомления ещё не было).